Övning Klasser och Objekt

Tänk noga igenom vilket publikt gränssnitt klassen du skriver ska ha gentemot övriga klasser, d.v.s. vilka publika metoder som ska finnas.
Använd inkapsling, med hjälp nyckelordet private, av datamedlemmar samt av metoder som är för internt bruk inom klassen.

Skriv för varje uppgift en main-metod där du skapar objekt av klassen och testar funktionaliteten.
Tips: Hårdkoda värden på argument till metoder och liknande i ditt main (istället för att läsa in med Scanner) så sparar du tid.

Övningsuppgifter

Uppgift 1, Tärning

a) Skriv en klass, Dice, som modeller en tärning.
En tärning vänder alltid en viss sida upp (ett tal 1 - 6). Detta representeras med en datamedlem, som jag kallar value.
En tärning kan kastas och då slumpas en ny sida fram. Det ska vara möjligt att skriva ut information om tärningen snyggt. Vi behöver alltså följande metoder:

- En konstruktor. Låt en nyskapad tärning vända sidan 1 upp.
- En metod, throwDice, för att kasta tärningen, d.v.s. slumpa ett nytt värde.
- En metod, getValue, för att avläsa värdet på en tärning.
- En metod, toString, för att få textinformation om tärningens tillstånd.

För att slumpa tal kan man använda sig av Math.random() som returnerar ett slumptal i intervallet [0.0..1.0[. Heltal i intervallet 1 – 6 fås med `value = (int)(Math.random()*6)+1;`

Ett exempel-main för att testa del a:

```
public static void main(String[] args) {
    Dice d1 = new Dice();
    Dice d2 = new Dice();
    System.out.println(d1.toString() + ", " + d2.toString()); // 1, 1
    d1.throwDice();
    d2.throwDice();
    System.out.println(d1.toString() + ", " + d2.toString()); // random
}
```

b) Skriv också ett main där 2 tärningar kastas ett antal gånger och resultatet skrivs ut. Skriv även kod som kastar en tärning 1000 gånger och beräknar frekvensen för de olika sidorna.

Uppgift 2, Account

Hämta filen Account3.java, som innehåller definitionen av bankkonto, från föreläsningen. Studera klassen. Lägg till eller ändra följande:

- En privat datamedlem, typ `long`, som representerar kontonumret. Ej ändringsbar.
- Ändra konstruktorerna så att ett objekt ges värde på kontonumret när det skapas.
- Lägg till en accessmetod för att avläsa kontonumret.
- Uppdatera metoden toString().

Skriv en main-metod där du skapar några olika Account-objekt och testar, t.ex.:

```
Account3 acc = new Account3("Anders", 123456, 1000.0);
long accNo = acc.getNumber();
System.out.println("account number: " + accNo);
System.out.println(acc.toString());
```

Output:

```
account number: 123456
name: Anders, number: 123456, balance: 1000,00
```

Uppgift 3, Patient

Skriv en klass, Patient, som representerar en patient med en journal. Klassen ska ha tre privata datamedlemmar för namn (String), personnummer (string) samt en journal (String).

I kontraktet ingår att

- namn och personnummer ska inte kunna ändras efter att objektet skapats,
- det ska gå att lägga till en text/anteckning till journalen – varje gång denna metod anropas ska även datumet för den nya anteckningen läggas till.
  Använd klassen LocalDate för att representera datumet (importera java.time.LocalDate) och använd klassmetoden LocalDate.now() för att genrera ett objekt med aktuellt datum.

Konstruktorn ska ha argument för namn och personnummer; journaltexten ska initialt sättas till ”” (en tom textsträng).

I övrigt ska klassen ha access-metoder för alla datamedlemmar samt en egen version av metoden toString.

Skriv även en main-metod där du skapar objekt av typen Patient och testar funktionaliteten.

Uppgift 4, Todo

Skriv en klass vars objekt representerar information om en uppgift som ska utföras, Todo. Ett objekt av typen Todo ska ha privata datamedlemmar för

- titel (String),
- en beskrivning (String),
- en tidpunkt då objektet senast uppdaterades (java.time.LocalDateTime),
- en flagga (boolean) som anger om uppgiften är utförd eller ej.

Konstruktorn ska ha argument för titel och beskrivning. Tiden för senaste uppdatering ska vara tiden då objektet skapas, vilket fås med java.time.LocalDateTime.now()[1]. Flaggan sätts till false (ej utförd).

I kontraktet för klassen ingår att

- titeln ska ej kunna ändras efter att objektet skapats,
- det ska gå att ändra flaggan som markerar om uppgiften är utförd till true,
- det ska vara möjligt att lägga till text till beskrivningen och datum för senaste uppdatering ska då uppdateras.

Det ska finnas access-metoder för datamedlemmarna samt en egen version av metoden toString.

Skriv sedan en main-metod som skapar objekt av typen Todo och testar funktionaliteten.

Uppgift 5, Todo med prioritet

Lägg till att Todo-objekt kan ha prioritet. För detta behöver du skapa en enum, Priority, med värden LOW, MEDIUM och HIGH.

Uppdatera klassen Todo med en datamedlem för prioritet. Prioriteten ska inte kunna ändras efter objektet skapats.

Denna version av klassen ska ha två konstruktorer, en som tar titel, beskrivning och prioritet som argument samt en som endast tar titel och beskrivning som argument och sätter prioriteten till medium (den senare konstruktorn kan om du vill anropa den första med this(…)).

Lägg även till en accessmetod för prioriteten och uppdatera toString.

Skriv sedan en main-metod där du testar att skapa objekt med båda konstruktorerna och anropa de nya metoderna.

Uppgift 6, Räkna med bråk

Innan du börjar med denna uppgift bör du gå igenom klassen Complex, från föreläsningen.

Skriv en klass Rational, som lagrar information om ett bråk (rationellt tal), dvs. två heltal, täljare och nämnare.

Skriv konstruktorer så att man kan skapa ett bråk med två, en eller ingen parameter (i sista fallet lagras 0/1). Notera att nämnaren aldrig får vara 0.

Skriv metoderna

- toString() så att en snygg utskrift fås,
- accessmetoder,
- metoden, `add(Rational r)` som adderar r till aktuellt bråk och returnerar ett nytt bråk, summan[2],

---

[1] För den vetgirige finns fullständig information om funktionaliteten för klassen LocalDateTime på https://download.java.net/java/early_access/panama/docs/api/java.base/java/time/LocalDateTime.html

[2] $\frac{a}{b}+\frac{c}{d}=\frac{ad+bc}{bd}$

- En variant, add(int number) som adderar ett heltal till aktuellt bråk och returnerar ett nytt.

Testa din kod i en main-metod (förslag finns nedan).

Uppgift 7, Mera bråk

Det är lämpligt att ett bråk alltid representeras på förenklad form (till exempel 10/15 => 2/3).

För att på ett enkelt sätt alltid hålla bråket på enklaste form kan du skriva en privat hjälpmetod som dividerar täljare och nämnare med största gemensamma delare. Största gemensamma delare kan till exempel beräknas enligt exemplet nedan.

```
private int gcd( int m, int n )  { // Greatest Common Divisor
  m = Math.abs( m );
  n = Math.abs( n );
  while( m != n ) {
    if( m > n )
      m -= n;
    else
      n -= m;
  }
  return m;
```

I vilka metoder behöver vi anropa gcd?

Förslag på testkod (main):

```
Rational r1 = new Rational(3, 9);
Rational r2 = new Rational(5, 30);
Rational r3 = new Rational(7);
System.out.println("r1 = " + r1);
System.out.println("r2 = " + r2);
System.out.println("r2 = " + r3);

System.out.println("r1, numerator: " + r1.getNumerator());
System.out.println("r1, denominator: " + r1.getDenominator());

Rational r4 = r1.add(r2);
System.out.println("r1 + r2 = " + r4);
```

Output:

```
r1 = 1/3
r2 = 1/6
r2 = 7/1
r1, numerator: 1
r1, denominator: 3
r1 + r2 = 1/2
```

Uppgift 8, En kö

Innan du börjar med denna uppgift bör du gå igenom Stack från föreläsningen.

En kö, Queue, är en datastruktur där element läggs till i ena änden (enqueue) och plockas ut i andra änden (dequeue), dvs. när ett element tas bort från kön är det alltid det som legat längst i kön.

Skriv klassen Queue där texter lagras i en privat array. Om den interna arrayen är full när ett nytt element läggs till, med enque, ska den utökas ”automatiskt” av din kod.

Klassen ska ha nedanstående publika metoder.

- Två konstruktorer, en som tar ett argument, utgångsstorleken på den interna arrayen samt parameterlös konstruktor som skapar en intern array som har plats för 10 element.
- size, som returnerar aktuellt antal element (inte detsamma som arrayens storlek)
- isEmpty, som returnerar true om kön är tom
- enqueue, som lägger till ett nytt element sist i kön. Om den interna arrayen är full ska en ny, med dubbel storlek, skapas och elementen kopieras över till denna – att utöka arrayen ska utföras av en privat hjälpmetod, resize.
- dequeue, som returnerar, och tar bort, det element som legat längst tid i kön. Notera att du förmodligen behöver packa den interna arrayen efter att ett element tagits bort, så att det inte finns ”hål” i listan.
- peek, som returnerar en referens till det element som legat längst tid i kön, utan att ta bort detta.
- getAll, som returnerar en array med de element som finns i kön; denna array ska vara en *kopia* av den interna arrayen
- toString, som returnerar en textrepresentation av kön (användbart när klassen testas).

Skriv också en klass med en main-metod som testar köns funktionalitet, t.ex.:

```
Queue q = new Queue(4);
q.enqueue("Buy a textbook");
q.enqueue("Follow the lectures");
q.enqueue("Read the textbook");
q.enqueue("Write code");
System.out.println(q.toString());

q.dequeue();
q.enqueue("Do it again");
System.out.println(q.toString());

String s = q.dequeue();
System.out.println("Dequeued: " + s);

s = q.peek();
System.out.println("Peeked at: " + s);

while(!q.isEmpty()) {
    System.out.println(q.dequeue());
}
System.out.println(q.toString());
```

Output:

```
[Buy a textbook;Follow the lectures;Read the textbook;Write code;]
[Follow the lectures;Read the textbook;Write code;Do it again;]
Dequeued: Follow the lectures
Peeked at: Read the textbook
Read the textbook
Write code
Do it again
[]
```

Om du vill lösa ett svårare problem kan du försöka att implementera så att den interna arrayen lagrar elementen cirkulärt (du behöver då inte packa arrayen när ett element tas bort, men du måste i stället hålla reda på var kön börjar och slutar, den fortsätta efter arrayens slut till tomma platser i början av denna).

Uppgift 9, Gissa talet

Skriv en klass GuessMyNumberModel som representerar data och logik för leken ”gissa ordet”. Klassen ska endast representera och data och metoder för leken, inte något om kommunikationen med användaren; vi ska alltså ha olika klasser för ”modellen” respektive kommunikationen med användaren, ett exempel på hög kohesion.

Det ska räcka med datamedlemmar för det hemliga talet, senaste gissningen, antalet gissningar, övre gränsen för det hemliga tal som ska slumpas samt ett java.util.Random-objekt.

Klassen ska följande metoder:

- en konstruktor med ett argument för övre gränsen för det slumpade talet,
- initGame, som slumpar ett nytt hemligt tal och nollställer antal gissningar,
- access-metoder för datamedlemmarna,
- setGuess, med ett argument för den nya gissningen,
- compareGuess (inget argument), som returnerar ett heltal > 0 om gissningen är för hög, < 0 om den är för låg och 0 om gissningen är korrekt (jämför strcmp i C) samt
- compareGuessToStr, som returnerar texten ”to high”, ”to low” eller ”correct” enligt ovan.

Förslag på main-metod för att testa lösningen:

```
GuessMyNumberModel guessModel = new GuessMyNumberModel(100);
Scanner scan = new Scanner(System.in);

String cont = "y";
while (cont.toLowerCase().charAt(0) == 'y') {

    guessModel.initGame();
    int guess;
    do {
        System.out.print(
                "Make a guess [1.." + guessModel.getMAX() + "]: ");
        String numStr = scan.nextLine().trim();
        guess = Integer.parseInt(numStr);
        guessModel.setGuess(guess);
        System.out.println("Your guess is " +
                guessModel.compareGuessToStr());
        System.out.println("Number of guesses: " +
                guessModel.getNoOfGuesses());
    }
    while (guess != guessModel.getSecret());

    System.out.println("Congratulations!");

    System.out.print("Continue (y/n)? ");
    cont = scan.nextLine();
}
```

```
System.out.println("Bye bye.");
scan.close();
```